UNIT-5

Recenttrends-Variational Autoencoders, Transformers,GPT Applications: Vision, NLP, Speech

Variational AutoEncoders

Variational Autoencoders (VAEs) are type of generative model in machine learning that create new data similar to the input they are trained on. They not only compress and reconstruct data like traditional autoencoders but also learn a continuous probabilistic representation of the underlying features. This unique approach helps VAEs to generate new, realistic data samples that closely resemble the original input. In this article, we will see more about VAEs and its core concepts.

Architecture of Variational Autoencoder

VAE is a special kind of autoencoder that can generate new data instead of just compressing and reconstructing it. It has three main parts:

1. Encoder (Understanding the Input)

The encoder takes input data like images or text and learns its key features. Instead of outputting one fixed value, it produces two vectors for each feature:

- **Mean (μ):** A central value representing the data.
- **Standard Deviation (σ):** It is a measure of how much the values can vary.

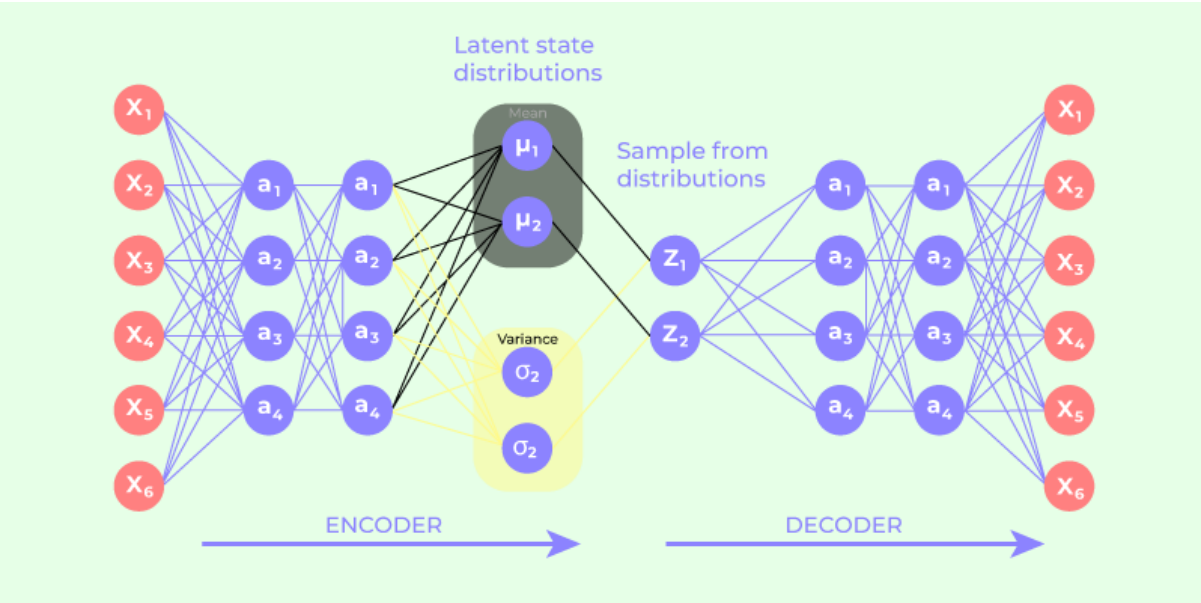
These two values define a range of possibilities instead of a single number.

2. Latent Space (Adding Some Randomness)

Instead of encoding the input as one fixed point it pick a random point within the range given by the mean and standard deviation. This randomness lets the model create slightly different versions of data which is useful for generating new, realistic samples.

3. Decoder (Reconstructing or Creating New Data)

The decoder takes the random sample from the latent space and tries to reconstruct the original input. Since the encoder gives a range, the decoder can produce new data that is similar but not identical to what it has seen.



Working Principle of Variational Autoencoders (VAEs)

The **working principle** of a Variational Autoencoder is based on how it **encodes**, **samples**, and **reconstructs** data in a **probabilistic** way to both **compress** and **generate** new data.

It mainly works in **three steps**:

1. Encoding (Learning the Distribution)

- The **encoder** takes the input data (like an image or text) and learns its **important features**.
- Instead of producing a single fixed vector, it outputs **two values** for each feature:
  - **Mean (μ)** → represents the center of the feature distribution.
  - **Standard deviation (σ)** → represents how much the features can vary.
- These two parameters define a **probability distribution** for each input instead of a single value.
- In other words, the encoder learns **how the input is distributed** in the latent space.

2. Sampling (Introducing Randomness in Latent Space)

- From the distribution defined by the mean and standard deviation, the model picks a **random point** (latent variable) using:

$$z = \mu + \sigma \times \epsilon$$

where $\epsilon$is a random noise sampled from a normal distribution $\mathcal{N}(0,1)$.

- This step is known as the **reparameterization trick**, which allows randomness but still keeps the model trainable.
- This randomness helps the VAE generate **new and slightly different versions** of the data each time.

3. Decoding (Reconstruction and Generation)

- The **decoder** takes the sampled latent variable $z$and tries to **reconstruct the original input**.
- Because the encoder gives a range of possible representations, the decoder can create **new outputs** that look similar to the training data but are not exact copies.
- During training, the VAE learns to make these reconstructions as close as possible to the original input.

4. Training Objective (Balancing Reconstruction and Regularization)

The VAE uses a **loss function** with two parts:

1. **Reconstruction Loss** – measures how close the output is to the original input.
2. **KL Divergence Loss** – ensures the learned latent space stays close to a standard normal distribution $\mathcal{N}(0,1)$.

The total loss is the sum of these two, helping the model to:

- Reconstruct data accurately.
- Learn a smooth, continuous latent space for generating new data.

In Simple Terms:

1. **Encoder** → Learns feature distributions (μ, σ)
2. **Sampler** → Picks a random point $z$using μ and σ
3. **Decoder** → Reconstructs the input or generates new data from z

Example (Analogy):

Imagine the encoder describing a face as:

- Mean = average features (eyes, nose, mouth position)
- Standard deviation = how much each feature can vary

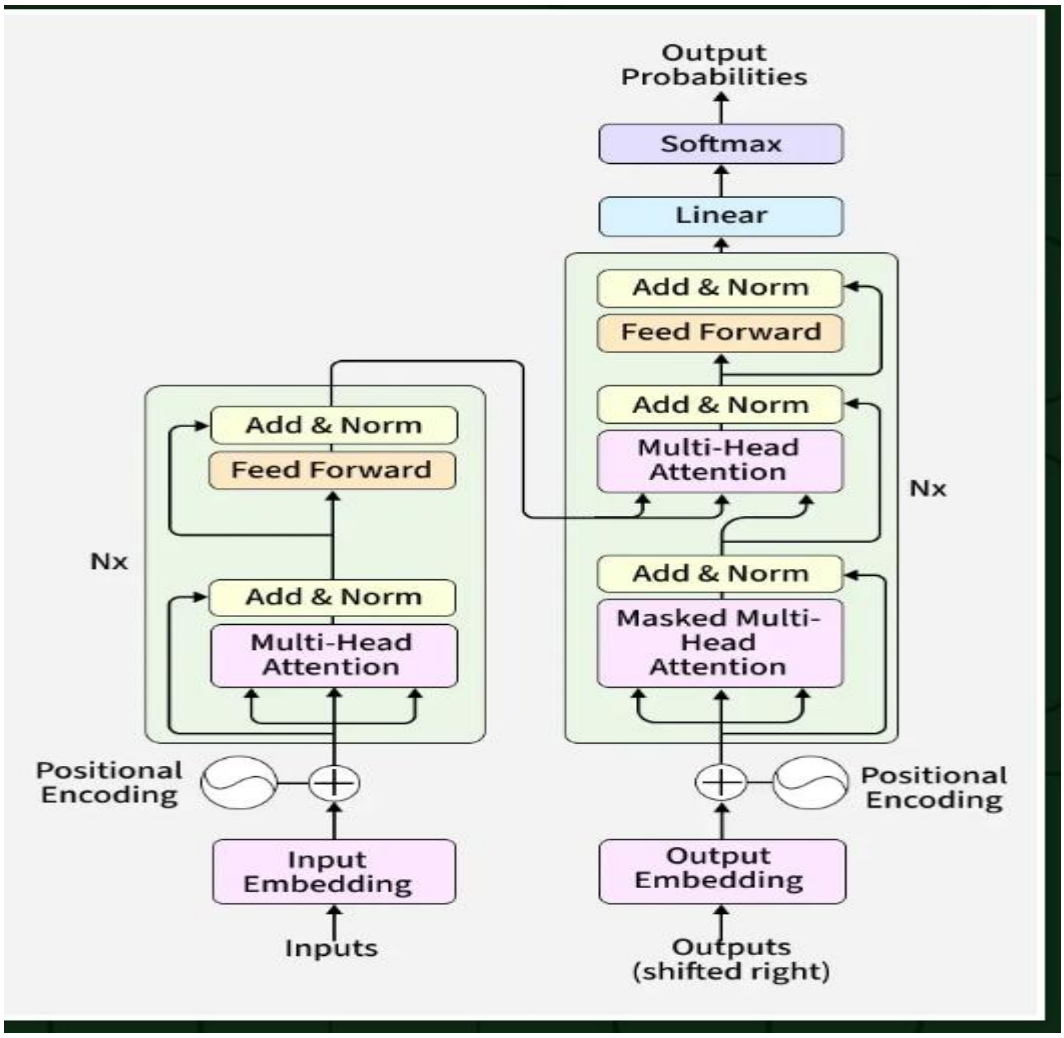
When sampling, the model slightly changes these features (like changing the smile or eye shape) — and the decoder then generates a **new face** that looks realistic but not identical to any seen before.

Architecture and Working of Transformers in Deep Learning

Transformers are a type of deep learning model that utilizes self-attention mechanisms to process and generate sequences of data efficiently. They capture long-range dependencies and contextual relationships making them highly effective for tasks like language modeling, machine translation and text generation. Transformer model is built on encoder-decoder architecture where both the encoder and decoder are composed of a series of layers that utilize self-attention mechanisms and feed-forward neural networks. This architecture enables the model to process input data in parallel making it highly efficient and effective for tasks involving sequential data.

- The encoder processes input sequences and creates meaningful representations.
- The decoder generates outputs based on encoder representations and previously predicted tokens.

The encoder and decoder work together to transform the input into the desired output such as translating a sentence from one language to another or generating a response to a query.



Encoder Side

1. Inputs

- Raw tokens (words or subwords) from the input sequence.
- These are mapped to numerical indices using a vocabulary.

2. Input Embedding

- Converts token indices into dense vectors of fixed size.
- These embeddings capture semantic meaning.

3. Positional Encoding

- Adds position-specific information to embeddings.
- Since Transformers lack recurrence, this helps preserve token order.
- Typically uses sine and cosine functions of different frequencies.

4. Encoder Layer (Nx repeated blocks)

Each encoder layer contains:

a. Multi-Head Attention

- Computes attention scores between all token pairs.
- Multiple heads allow learning different relationships in parallel.

b. Add & Norm

- Residual connection: adds input to output of attention.
- Layer normalization stabilizes training.

c. Feed Forward

- A two-layer fully connected network applied to each token independently.
- Adds non-linearity and depth.

d. Add & Norm

- Another residual connection and normalization after feed-forward.

Decoder Side

1. Outputs (Shifted Right)

- During training, the output sequence is shifted right to prevent the model from seeing future tokens.
- This enables autoregressive generation.

2. Output Embedding

- Similar to input embedding: maps output tokens to dense vectors.

3. Positional Encoding

- Same as in the encoder: adds positional information to output embeddings.

4. Decoder Layer (Nx repeated blocks)

Each decoder layer contains:

a. Masked Multi-Head Attention

- Prevents attention to future tokens.
- Ensures predictions are based only on known past outputs.

b. Add & Norm

- Residual connection and normalization after masked attention.

c. Multi-Head Attention (with Encoder Output)

- Attends to encoder outputs to incorporate input context.
- Enables alignment between input and output sequences.

d. Add & Norm

- Residual connection and normalization after cross-attention.

e. Feed Forward

- Same as encoder: two-layer network applied to each token.

f. Add & Norm

- Final residual connection and normalization.

Final Output

1. Linear Layer

- Projects decoder outputs to vocabulary size.
- Converts hidden states into logits for each token.

2. Softmax Layer

- Converts logits into probabilities.
- Highest probability token is selected as the output.

Generative Pre-trained Transformer (GPT)

Generative Pre-trained Transformer (GPT) is a large language model that can understand and produce human-like text. It works by learning patterns, meanings and relationships between words from massive amounts of data. Once trained, GPT can perform various language-related tasks such as writing, summarizing, answering questions and even coding all from a single model.

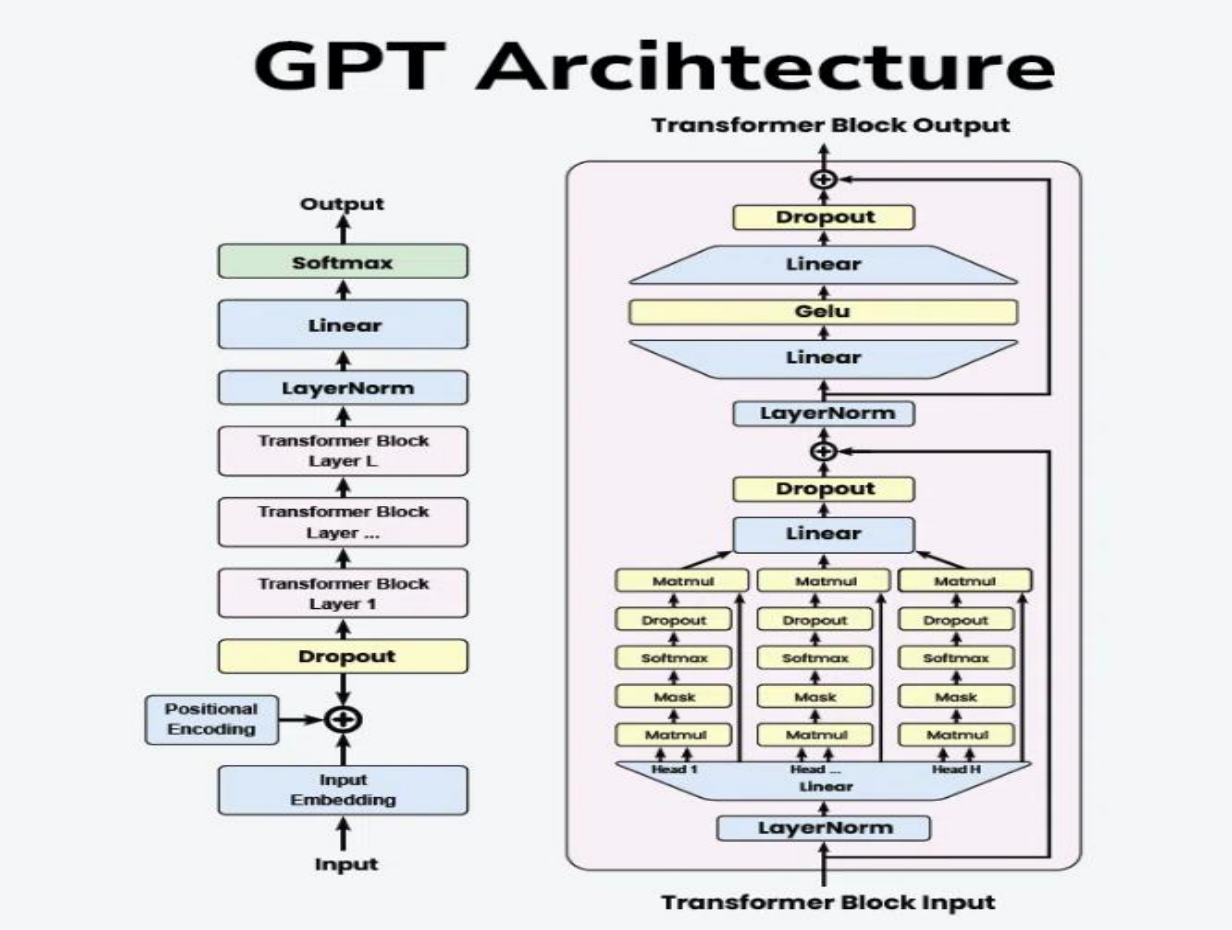
How GPT Works

GPT models are built upon the transformer architecture, introduced in 2017, which uses self-attention mechanisms to process input data in parallel, allowing for efficient handling of long-range dependencies in text. The core process involves:

1. **Pre-training:** The model is trained on vast amounts of text data to learn language patterns, grammar, facts and some reasoning abilities.
2. **Fine-tuning:** The pre-trained model is further trained on specific datasets with human feedback to align its responses with desired outputs.

This two-step approach enables GPTs to generate coherent and contextually relevant responses across a wide array of topics and tasks.

Architecture



1. Input Embedding

- **Input**: The raw text input is tokenized into individual tokens (words or subwords).
- **Embedding**: Each token is converted into a dense vector representation using an embedding layer.

2. Positional Encoding: Since transformers do not inherently understand the order of tokens, positional encodings are added to the input embeddings to retain the sequence information.

3. Dropout Layer: A dropout layer is applied to the embeddings to prevent overfitting during training.

4. Transformer Blocks

- **LayerNorm**: Each transformer block starts with a layer normalization.
- **Multi-Head Self-Attention**: Multi-Head Self-Attention are core component where the input passes through multiple attention heads.
- **Add & Norm**: The output of the attention mechanism is added back to the input (residual connection) and normalized again.
- **Feed-Forward Network**: A position-wise Feed-Forward Network is applied, typically consisting of two linear transformations with a GeLU activation in between.
- **Dropout**: Dropout is applied to the feed-forward network output.

5. Layer Stack: The transformer blocks are stacked to form a deeper model, allowing the network to capture more complex patterns and dependencies in the input.

6. Final Layers

- **LayerNorm**: LayerNorm is final layer normalization is applied.
- **Linear**: The output is passed through a linear layer to map it to the vocabulary size.
- **Softmax**: A Softmax layer is applied to produce the final probabilities for each token in the vocabulary.

Compare CNN-based vision models with Transformer-based vision models (e.g., ViT).

Introduction

Vision Transformers (ViTs) have emerged as a powerful alternative to Convolutional Neural Networks (CNNs) for image processing tasks, challenging the dominance of CNNs. For students and practitioners in AI and computer vision, understanding the differences between these two architectures is critical. This blog post explores how Vision Transformers compare to CNNs, highlighting their key differences, advantages, limitations, and applications.

Features	Vision Transformers (ViT)	Convolutional Neural Network (CNN)
Feature Extraction	Uses self-attention across image patches	Uses convolutional filters to extract local features
Global Context	Captures global relationships between patches in a single layer	Requires deep layers to build global understanding from local features
Image Processing	Divides the image into non-overlapping patches	Scans the image using overlapping receptive fields (filters)
Positional Encoding	Requires explicit positional encoding for patch location awareness	Implicitly encodes position through convolution structure
Model Complexity	Higher computational cost due to attention mechanism	Typically less computationally expensive than ViTs
Data Requirements	Requires large datasets to perform well	Performs well on both small and large datasets
Scalability	Highly scalable with larger datasets and bigger models	Scales well, but prone to overfitting on small datasets
Inductive Biases	Minimal inductive bias; learns relationships directly from data	Strong inductive bias toward local features (edges, textures)
Performance on Small Datasets	Tends to overfit on small datasets	Performs better on smaller datasets due to built-in bias
Interpretability	Difficult to interpret due to attention mechanism	Easier to interpret through learned filters (e.g., edge detectors)

Applications of Transformers and GPT models in Vision, NLP, and Speech processing.

Transformers and GPT (Generative Pre-trained Transformer) models have become the foundation of modern Artificial Intelligence.
They are used in **Natural Language Processing (NLP)**, **Computer Vision (CV)**, and **Speech Processing** due to their ability to handle sequential data efficiently using **self-attention mechanisms**.

1. Applications in Natural Language Processing (NLP)

Transformers and GPT models are most widely used in NLP tasks because they can understand context, meaning, and relationships between words.

Main Applications:

1. **Text Generation:**
    - GPT models can generate human-like text for chatbots, story writing, and content creation.
    - Example: ChatGPT, GPT-4, etc.
2. **Machine Translation:**
    - Used for converting text from one language to another.
    - Example: Google Translate uses transformer-based models.
3. **Text Summarization:**
    - Automatically shortens long documents into concise summaries while preserving meaning.
4. **Question Answering and Chatbots:**
    - GPT and BERT-like models power virtual assistants (e.g., Siri, Alexa, ChatGPT).
5. **Sentiment Analysis:**
    - Detects emotions or opinions (positive, negative, neutral) in reviews or social media posts.

2. Applications in Computer Vision

Although transformers were originally designed for text, **Vision Transformers (ViT)** and GPT-like architectures are now used for visual tasks.

Main Applications:

1. **Image Classification:**
    - Vision Transformers (ViT) classify images into categories like cats, dogs, cars, etc.
2. **Object Detection and Segmentation:**
    - DETR (DEtection TRansformer) uses transformers for identifying and locating objects in an image.
3. **Image Captioning:**

- Combines vision and language transformers to describe images in natural language.

4. **Generative Image Modeling:**
    - Models like **DALL·E** and **Stable Diffusion** (based on transformer principles) generate realistic images from text descriptions.
5. **Video Understanding:**
    - Video Transformers analyze sequences of frames for action recognition and scene understanding.

3. Applications in Speech Processing

Transformers and GPT-based architectures are also used in speech and audio tasks because they can model long-term dependencies in time-series data.

Main Applications:

1. **Automatic Speech Recognition (ASR):**
    - Converts spoken words into text.
    - Example: Whisper (by OpenAI) and Wav2Vec2.0 (by Facebook) use transformer-based models.
2. **Speech Synthesis (Text-to-Speech):**
    - GPT-like models generate natural human-like speech from text.
    - Example: Tacotron 2 and VALL-E (speech generation models).
3. **Speaker Identification:**
    - Identifies or verifies a person's voice using transformer embeddings.
4. **Speech Translation:**
    - Directly converts speech in one language to speech/text in another language.
5. **Emotion Recognition:**
    - Detects emotion or tone in speech (happy, sad, angry, etc.).